

Voronoi Diagram Encoded Hashing

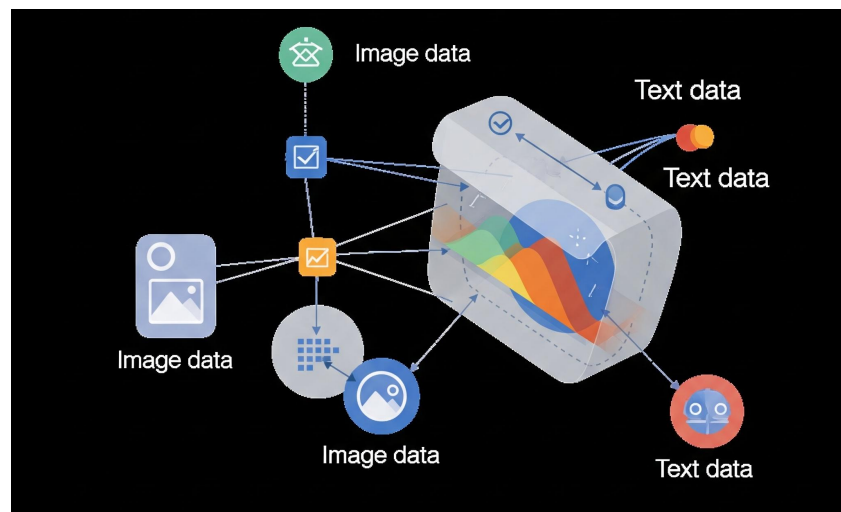
Yang Xu and Kai Ming Ting

State Key Laboratory for Novel Software Technology, Nanjing University, 210023, China

School of Artificial Intelligence, Nanjing University, 210023, China



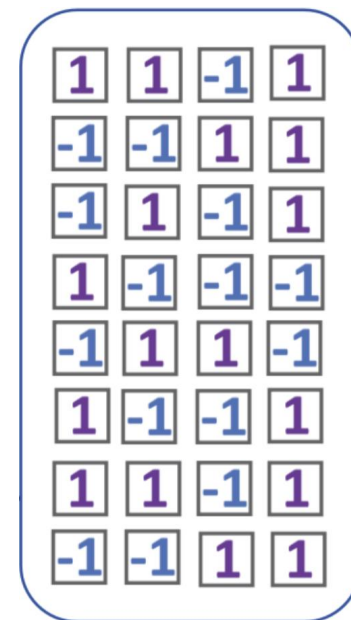
Motivation: The Goal of Hashing



High-dimensional data

Euclidean distance

L2H



Compact binary codes

Hamming distance



Two Doubts on the "Learning" in L2H

- **Doubt 1: Is complex learning necessary?**

A recent study showed that a simple, no-learning LSH variant can achieve comparable accuracy to complex L2H methods, but much faster.

- **Doubt 2: Are we constrained by our tools?**

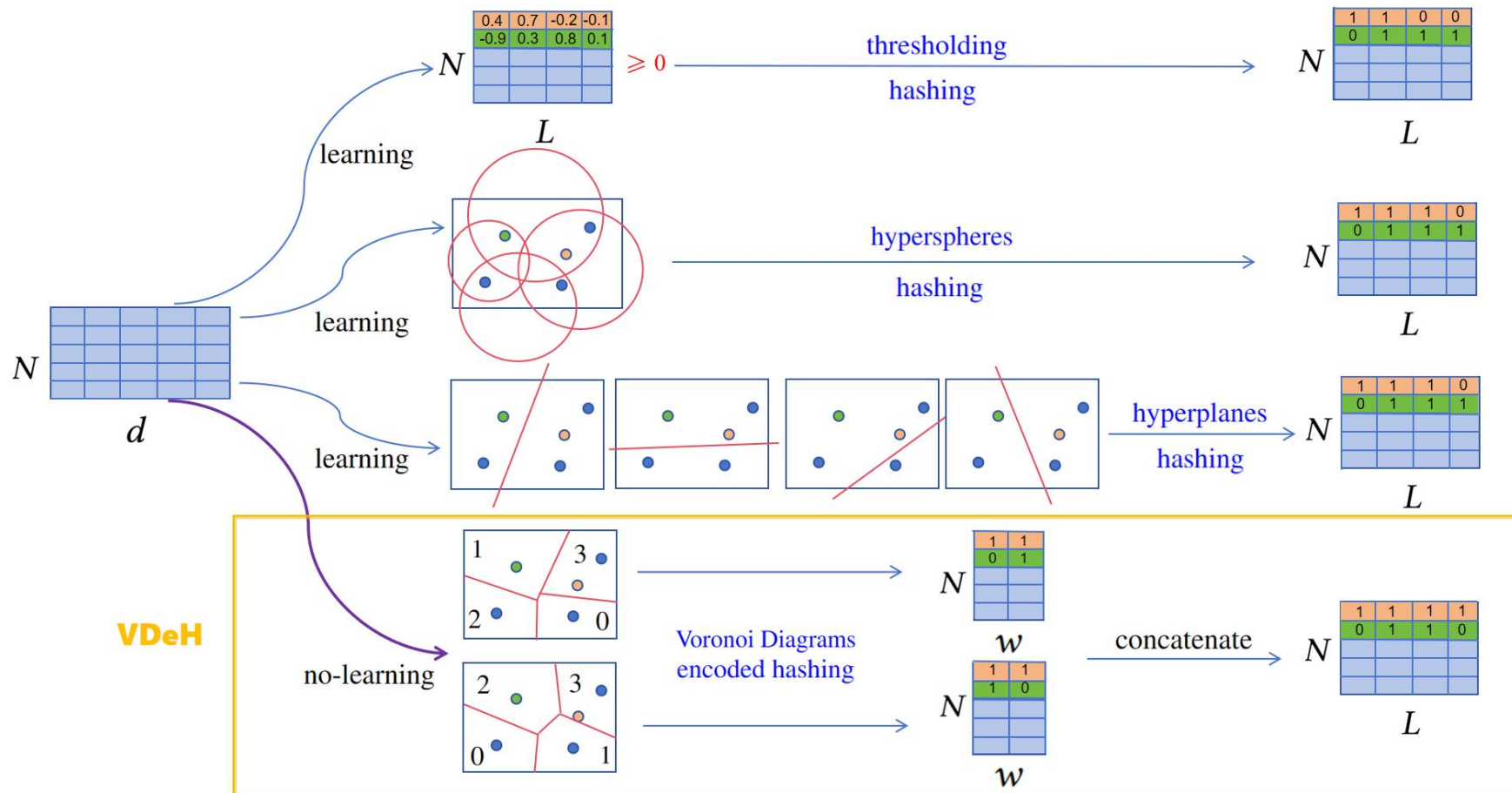
- Existing L2H methods are limited to just three types of hash functions: **thresholding**, **hyperspheres**, and **hyperplanes**.
- These functions require a complex, costly learning phase to work well.

- There are three widely claimed key properties for effective binary hashing:
 - **Full Space Coverage:** Every data point can be assigned a hash code. Avoids retrieval failures.
 - **Entropy Maximization:** Data is distributed evenly across all hash buckets. Maximizes information and avoids under-utilized codes.
 - **Bit Independence:** Each bit in the code provides unique information. Eliminates redundancy and improves efficiency.
- Existing methods use learning to enforce these properties.

Our Core Insight: Voronoi Diagrams

- **The Insight:** Voronoi diagrams inherently possess these three properties, **without any learning**.
- **Natural Full Space Coverage:** The cells collectively partition and cover the entire space.
- **Natural Entropy Maximization:** When generated from data samples, each cell tends to cover a similar number of points.
- **Achievable Bit Independence:** It is feasible to transform a Voronoi diagram into a set of mutually independent hash functions.

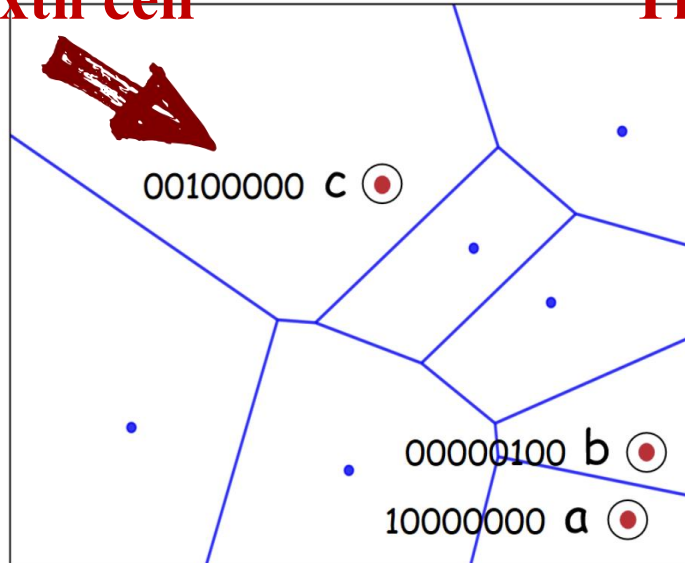
Voronoi Diagram Encode Hashing



How VDeH Works: The Encoding is Key

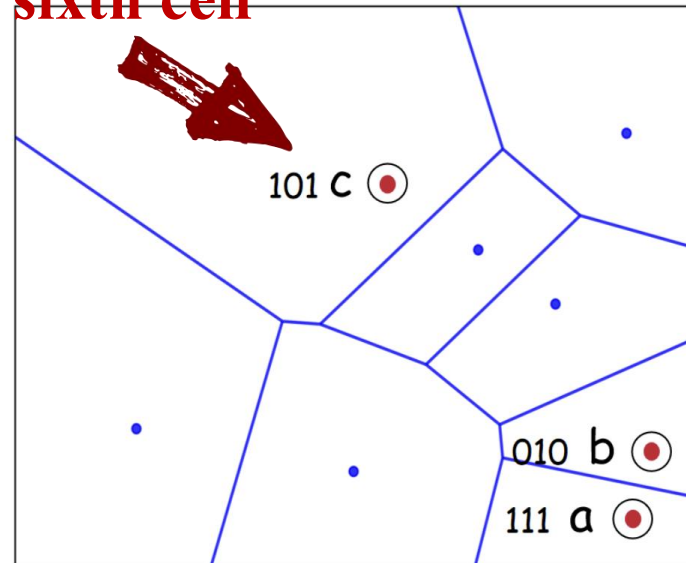
- **Challenge:** A point x can only be in one Voronoi cell. A direct 1-of- k mapping ($h_i(x)=1$, others are 0) creates **dependent** bits.
- **Solution: Encoded Hashing**

The sixth cell



(a) Sparse 8-bit embedding

The sixth cell



(b) Dense 3-bit embedding

The VDeH Algorithm

- **Step 1: Partition**

1. Given a dataset $\mathcal{X}$, we randomly sample ψ points to form a set $\mathcal{D}$, and generate the set of Voronoi diagram hash functions $H = \{h_1, h_2, \dots, h_\psi\}$. According to Eq.(3), there is one and only one $h_i(\mathbf{x}) = 1$ for any point $\mathbf{x}$, and $\forall u \neq i, h_u(\mathbf{x}) = 0$.

- **Step 2: Locate & Index**

2. The ‘raw’ ψ -bit vector $[h_1(x), h_2(x), \dots, h_\psi(x)]$, with $\psi \leq 2^w$, is encoded as a new binary vector with w bits:

$$b(x) = [e_1(x), e_2(x), \dots, e_w(x)],$$

$$e_k(x) = L_{k-1} \bmod 2, \text{ where } L_0 = [\arg_{l \in [1, \psi]} h_l(x) = 1] - 1 \text{ and } L_k = \lfloor \frac{L_{k-1}}{2} \rfloor.$$

This encoding converts the ‘raw’ ψ -bit vector into its dense multi-bit binary representation, effectively converting an integer into a binary number.

The VDeH Algorithm

- **Step 3: Encode & Concatenate**

3. After repeating the above two steps L/w times, we concatenate all w -bit vectors $b(\mathbf{x})$ to formulate a L -bit code $\mathcal{B}(\mathbf{x}) = [b^1(\mathbf{x}), \dots, b^{L/w}(\mathbf{x})]$ for any point $\mathbf{x}$.

Experimental Setup

- **Datasets**
 - **Images:** CIFAR-10 , GIST
 - **Text:** Nytimes , Kosarak (more challenging)
- **Baselines (9 methods)**
 - **Thresholding-based:** SH, CBE, ITQ, BP, SP
 - **Hyperspheres-based:** SpH
 - **Hyperplanes-based:** DSH, SELVE, rcLSH
- **Metrics**
 - Mean Average Precision (mAP)
 - Precision-Recall (PR) Curve

Results: Superior Accuracy

Dataset	bits	VD-based	HS-based	HP-based			Th-based				
		VDeH	SpH	rcLSH	DSH	SELVE	SH	CBE	ITQ	BP	SP
CIFAR-10	128	.366	.242	.283	.318	.199	.117	.315	.355	.344	.352
	256	.438	.289	.351	.372	<u>.176</u>	.153	.342	.377	<u>.343</u>	.370
	512	.468	.304	.392	.401	<u>.167</u>	.193	.396	.389	<u>.339</u>	.397
	1024	.501	.314	.412	.429	<u>.169</u>	.201	.401	.398	<u>.332</u>	.407
	2048	.525	.321	.432	.457	<u>.169</u>	.214	.407	.401	<u>.335</u>	.426
GIST	128	.664	.222	.582	.501	.430	.338	.617	.644	.645	.604
	256	.714	.223	.631	.580	<u>.399</u>	.454	.644	.650	<u>.641</u>	.614
	512	.763	.232	.651	.618	<u>.402</u>	.501	.656	.654	<u>.634</u>	.638
	1024	.774	.233	.659	<u>.610</u>	<u>.399</u>	.514	.657	.657	<u>.642</u>	.638
	2048	.793	.242	.687	.621	<u>.398</u>	.532	.662	.663	<u>.644</u>	.641
Nytimes	128	.116	.100	.017	.016	.003	.033	.022	.023	.023	.022
	256	.165	.110	.026	.033	.004	.055	.029	.029	.029	.027
	512	.340	.146	.028	.185	<u>.004</u>	.160	<u>.026</u>	.030	<u>.026</u>	.029
	1024	.348	<u>.088</u>	.103	.339	.006	.295	.108	.104	<u>.008</u>	.106
	2048	.376	<u>.089</u>	.113	.326	.008	.310	.100	<u>.097</u>	<u>.008</u>	<u>.100</u>
Kosarak	128	.457	.256	.235	.368	.203	.375	.255	.261	.254	.258
	256	.603	.296	.271	.494	.226	.512	.286	.293	.295	.304
	512	.607	.346	.308	<u>.437</u>	.279	.571	.301	.298	.346	.341
	1024	.615	.372	<u>.307</u>	<u>.415</u>	.289	<u>.569</u>	<u>.300</u>	.302	.351	.357
	2048	.638	.381	.313	<u>.412</u>	.300	.579	.309	.312	.359	.369

Results: Superior Accuracy

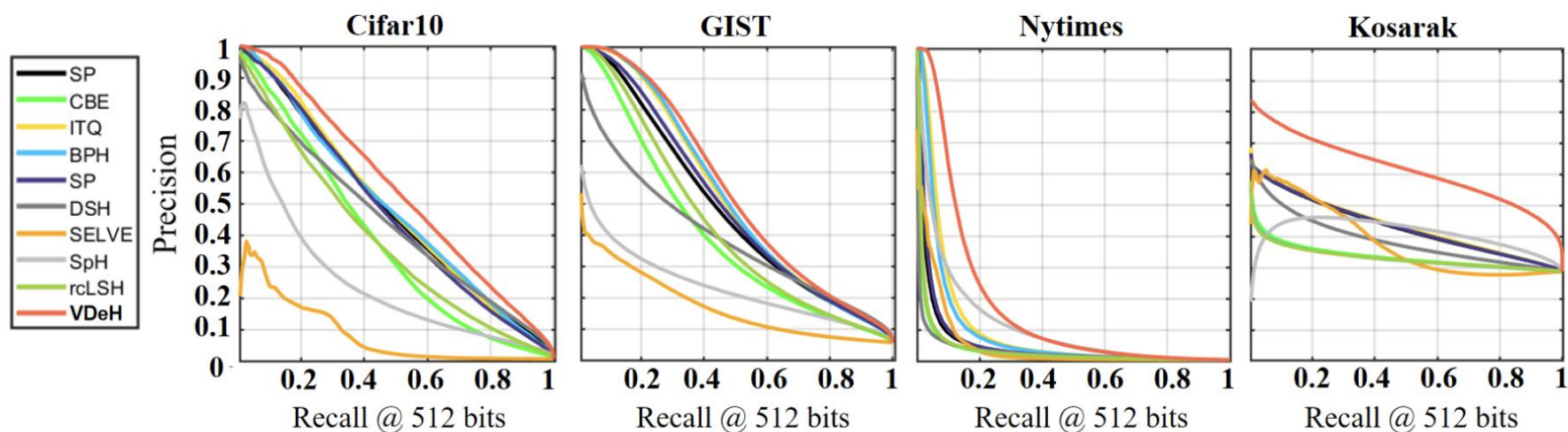


Fig. 2: PR curves on four datasets with the 512-bit binary codes

Results: Superior Accuracy

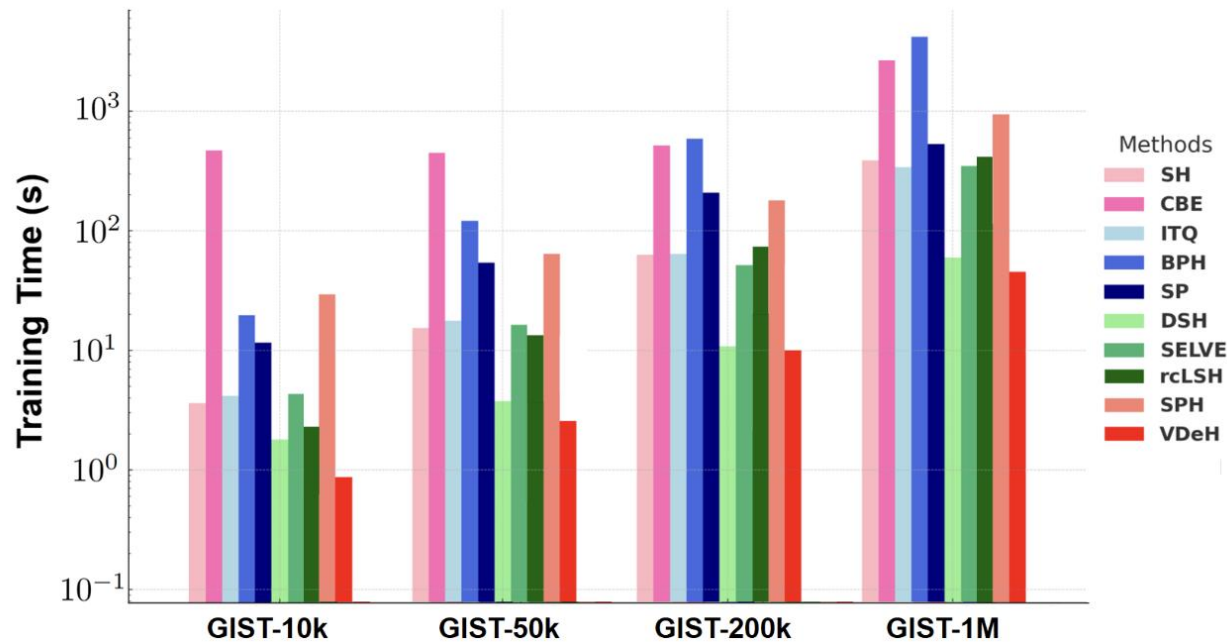


Fig. 4: Training time comparison on GIST dataset with the 512-bit binary codes.

- We introduced VDeH, a new no-learning data-dependent hashing method.
- **Key Contributions:**
 - **A New Perspective:** We identified that Voronoi diagrams naturally possess the three key properties that L2H methods struggle to learn.
 - **A Simple Method:** VDeH directly leverages these properties, creating a simple, fast, and data-dependent hashing scheme without complex learning.
 - **Demonstrated Superiority:** VDeH achieves state-of-the-art accuracy with significantly lower computational cost.

Thanks!

Q&A

